

BANCO OFICIAL DE PROYECTOS INTEGRADORES

VOLUMEN II · PROYECTOS 11 AL 20

Ingeniería de requerimientos predefinida · MVP Web + Móvil · Java 21 · Spring Boot · PostgreSQL ·
React/React Native + TypeScript · Docker · GitHub/CI · Spring AI

Documento de asignación académica para equipos de 3 a 5 estudiantes

Cada proyecto se considera definido y aprobado por el cliente para fines de la asignatura.

1. Propósito del Volumen II

Este volumen amplía el banco oficial con diez proyectos adicionales de complejidad equivalente. Cada proyecto se entrega al equipo como un encargo académico cerrado: los estudiantes no deben inventar el problema, sino analizarlo, modelarlo y construirlo con disciplina de Ingeniería de Software. El reto consiste en transformar requisitos ya acordados en una solución ejecutable, trazable, testeable y defendible individualmente.

La solución esperada es un monolito modular con arquitectura hexagonal pragmática, PostgreSQL, backend Java 21 + Spring Boot, cliente web React + TypeScript, cliente móvil React Native + TypeScript, Docker para el entorno, GitHub para colaboración e integración continua y un uso acotado de Spring AI sólo cuando aporte valor real y pueda controlarse.

2. Reglas comunes de ejecución

- Los equipos serán de 3 a 5 estudiantes y trabajarán sobre el mismo proyecto durante toda la asignatura.
- La asignación interna de tareas no limita la responsabilidad académica: cada integrante debe conocer y defender todo el alcance evaluable del corte.
- Cada proyecto se implementa incrementalmente; ninguna entrega puede consistir sólo en documentación.
- En todas las entregas debe existir evidencia de base de datos, backend, web y móvil acorde al nivel de avance.
- Las reglas de negocio obligatorias no son opcionales ni pueden reemplazarse por decisiones de conveniencia del equipo.
- Las decisiones adicionales son válidas si no contradicen el alcance y quedan documentadas como supuestos o ADR.
- No se exigen microservicios. Se exige separación modular, puertos/adaptadores, contratos claros y disciplina de dependencias.
- La persistencia debe evolucionar mediante migraciones; no se acepta crear manualmente la base en cada máquina como procedimiento oficial.
- GitHub debe evidenciar trabajo colaborativo mediante issues, branches, commits coherentes, pull requests y revisiones.
- La aplicación móvil debe consumir la misma API del backend; no es una maqueta desconectada.
- Spring AI nunca reemplaza reglas de negocio deterministas, autorización ni validaciones críticas.
- Cada entrega será evaluada contra la delimitación del proyecto y la defensa individual correspondiente.

3. Estructura uniforme de las cuatro entregas

Parte	Momento	Propósito acumulativo	Mínimo técnico transversal
I	Parcial 1	Comprender el dominio y construir la base técnica verificable.	Requisitos trazables + modelo relacional + PostgreSQL + backend fundacional + primeras vistas web/móvil conectadas.
II	Trabajos prácticos integradores	Cerrar el primer flujo transaccional extremo a extremo.	Persistencia real + reglas + REST + pruebas + flujo web funcional + flujo móvil funcional + Git/CI inicial.

III	Parcial 2	Ampliar cobertura, seguridad, calidad e integración.	Módulos principales + permisos + errores + pruebas + web avanzada + múltiples flujos móviles + Docker/CI estable.
IV	Examen final	Entregar y defender el MVP completo.	MVP integrado + documentación final + datos demo + pruebas + observabilidad básica + IA acotada + defensa integral.

4. Contenido documental obligatorio por cada corte

- Portada, identificación del equipo, versión y fecha del corte.
- Matriz de alcance del corte: requisito incluido, estado, evidencia y responsable primario.
- Diagrama de contexto y actores actualizado.
- Casos de uso o historias de usuario del alcance, con criterios de aceptación.
- Reglas de negocio numeradas y trazables.
- Modelo entidad-relación actualizado y diccionario de datos de tablas relevantes.
- Diagrama de arquitectura/paquetes y explicación de responsabilidades.
- Catálogo de endpoints implementados con request/response/error principales.
- Mapa de pantallas web y móvil del corte.
- Estrategia y evidencia de pruebas.
- Evidencia de colaboración Git/GitHub y decisiones técnicas relevantes.
- Registro de riesgos, deuda técnica y pendientes para el siguiente corte.

5. Requisitos no funcionales comunes

Código	Requisito
RNF-01	Aplicar autenticación y autorización básica por roles cuando el flujo lo requiera.
RNF-02	Validar entradas en backend y devolver errores HTTP coherentes.
RNF-03	Mantener separación entre dominio/aplicación e infraestructura.
RNF-04	Usar PostgreSQL como fuente de verdad transaccional.
RNF-05	Gestionar esquema mediante migraciones versionadas (por ejemplo Flyway).
RNF-06	Evitar exposición innecesaria de datos sensibles en logs y respuestas.
RNF-07	Incluir auditoría básica en operaciones críticas.
RNF-08	Implementar pruebas unitarias de reglas y pruebas de integración de flujos críticos.

RNF-09	Usar TypeScript sin abuso de any en web y móvil.
RNF-10	Mantener contratos API tipados del lado cliente.
RNF-11	Empaquetar entorno reproducible con Docker/Compose cuando corresponda.
RNF-12	Automatizar build/test básicos mediante CI en GitHub.
RNF-13	Documentar decisiones y supuestos de diseño.
RNF-14	Proveer datos de demostración controlados y reproducibles.
RNF-15	Manejar estados de carga, vacío y error en interfaces.
RNF-16	Incluir observabilidad básica: logs útiles, correlation/request id si se implementa y manejo centralizado de errores.

11. FarmaStock 360 - Gestión Operativa de Farmacia, Inventario por Lotes y Ventas

A. Situación del cliente

Una farmacia de tamaño mediano necesita reemplazar registros dispersos en hojas de cálculo y cuadernos por una plataforma que controle productos, lotes, fechas de vencimiento, proveedores, compras, ventas, devoluciones y alertas de stock. El cliente ha aclarado que el sistema es administrativo y comercial: no debe diagnosticar, prescribir ni recomendar tratamientos.

B. Objetivo contractual del producto

Construir un MVP web y móvil que permita operar inventario farmacéutico por lote, registrar abastecimiento y venta, mantener trazabilidad de movimientos y advertir oportunamente vencimientos y niveles mínimos, con roles diferenciados para administración, almacén y caja.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Configura usuarios, catálogos, parámetros, proveedores y consulta indicadores.
Encargado de almacén	Registra recepciones, lotes, ajustes y controla vencimientos/stock.
Cajero	Busca productos disponibles y registra ventas/devoluciones permitidas.
Supervisor	Revisa cierres, movimientos excepcionales y alertas.
Cliente	Actor externo de la venta; no requiere autenticación en el MVP.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Usuarios y roles
- Productos y categorías
- Proveedores
- Compras y recepción
- Lotes y vencimientos
- Inventario y movimientos
- Ventas y detalle
- Devoluciones controladas
- Alertas y tablero
- Auditoría básica

E. Reglas de negocio obligatorias

- RN-01. Todo ingreso de inventario debe quedar asociado a un lote y fecha de vencimiento cuando corresponda.

- RN-02. No se puede vender cantidad superior al stock disponible.
- RN-03. La salida de stock debe priorizar lotes con vencimiento más próximo (FEFO) para productos configurados como perecederos.
- RN-04. Un lote vencido queda bloqueado para venta.
- RN-05. Toda modificación manual de stock requiere motivo y usuario responsable.
- RN-06. Una venta confirmada no puede editarse; sólo puede anularse mediante operación trazable según permisos.
- RN-07. Los productos pueden estar activos o inactivos, pero los históricos nunca se eliminan físicamente.
- RN-08. Las devoluciones sólo pueden referenciar una venta previa y no superar la cantidad vendida.
- RN-09. El stock mínimo es configurable por producto y genera alertas.
- RN-10. El sistema no debe emitir recomendaciones médicas ni sustituir una receta.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Producto, CategoriaProducto, Proveedor, Compra, CompraDetalle, Lote, MovimientoInventario, Venta, VentaDetalle, Devolucion, AlertaStock, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar autenticación y roles básicos.
- RF-02 Registrar, editar, activar e inactivar productos.
- RF-03 Clasificar productos por categoría y presentación.
- RF-04 Gestionar proveedores.
- RF-05 Registrar compras con sus detalles.
- RF-06 Recepcionar compras creando lotes y existencias.
- RF-07 Consultar stock actual por producto y lote.
- RF-08 Registrar ajustes de inventario con motivo.
- RF-09 Detectar y listar lotes próximos a vencer.
- RF-10 Bloquear lotes vencidos para venta.
- RF-11 Buscar productos por nombre, código o categoría.
- RF-12 Registrar una venta con múltiples productos.
- RF-13 Descontar stock automáticamente al confirmar venta.
- RF-14 Consultar comprobante/resumen de venta.
- RF-15 Registrar devoluciones vinculadas a una venta.
- RF-16 Consultar kardex simplificado por producto.
- RF-17 Mostrar tablero de stock bajo, vencimientos y ventas.
- RF-18 Registrar auditoría de operaciones críticas.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Login y selección de rol
- Dashboard con KPIs operativos

- CRUD de productos/categorías
- CRUD de proveedores
- Recepción de compras
- Vista de lotes y vencimientos
- Inventario y kardex
- Punto de venta simplificado
- Devoluciones
- Alertas y auditoría

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Inicio con alertas prioritarias
- Consulta rápida de stock
- Escaneo/entrada manual de código de producto
- Detalle de lotes y vencimientos
- Registro de ajuste autorizado
- Consulta de ventas recientes
- Notificaciones internas de stock/vencimiento

J. Flujo crítico que debe demostrarse extremo a extremo

Administrador configura producto y proveedor → almacén registra una compra y recepción por lotes → el sistema actualiza stock → cajero busca productos y confirma una venta → se descuenta inventario aplicando FEFO → supervisor consulta movimientos, alertas y trazabilidad.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- POST /auth/login
- GET/POST /products
- GET/POST /suppliers
- POST /purchases
- POST /purchases/{id}/receive
- GET /inventory
- POST /inventory/adjustments
- GET /lots/expiring
- POST /sales
- GET /sales/{id}
- POST /sales/{id}/returns
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.

- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.
- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede usarse únicamente para resumir alertas operativas o generar una explicación textual del estado de inventario a partir de datos ya calculados por el backend. La IA no prescribe ni recomienda medicamentos.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.
- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.
- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- Explique cómo evita vender un lote vencido.
- ¿Dónde se implementa la regla FEFO y por qué?
- ¿Qué diferencia existe entre stock calculado y movimiento de inventario?
- ¿Cómo impediría una devolución mayor a lo vendido?
- Muestre la transacción que confirma una venta.
- ¿Qué endpoint consume la pantalla móvil de stock?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

12. BiblioSmart - Biblioteca, Préstamos, Reservas y Control de Ejemplares

A. Situación del cliente

Una biblioteca académica desea digitalizar catálogo, ejemplares físicos, usuarios, préstamos, devoluciones, reservas y sanciones operativas. Actualmente se confunden títulos con ejemplares y no existe trazabilidad fiable de quién tiene cada copia.

B. Objetivo contractual del producto

Construir un sistema web/móvil que gestione catálogo bibliográfico y ejemplares individualmente, controle disponibilidad, préstamos, renovaciones y reservas, y entregue a cada usuario una vista clara de su situación.

C. Actores y responsabilidades

Actor	Responsabilidad
Bibliotecario	Administra catálogo, ejemplares, préstamos y devoluciones.
Administrador	Gestiona parámetros, usuarios y reportes.
Lector	Consulta catálogo, reserva y revisa sus préstamos.
Supervisor	Consulta mora, sanciones y estadísticas.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Usuarios
- Autores y editoriales
- Catálogo bibliográfico
- Ejemplares
- Préstamos
- Devoluciones
- Reservas
- Renovaciones
- Sanciones
- Dashboard y auditoría

E. Reglas de negocio obligatorias

- RN-01. Un título puede tener varios ejemplares físicos con código único.
- RN-02. Sólo ejemplares disponibles pueden prestarse.
- RN-03. Cada préstamo debe tener fecha de salida y fecha prevista de devolución.
- RN-04. Una devolución cambia el ejemplar a disponible salvo que exista una reserva prioritaria.
- RN-05. Un lector con sanción activa no puede realizar nuevos préstamos.

- RN-06. La renovación se permite sólo si el ejemplar no está reservado por otro lector.
- RN-07. Una reserva vence luego del plazo parametrizado de retiro.
- RN-08. Los préstamos históricos no se eliminan.
- RN-09. La disponibilidad mostrada debe derivarse del estado real de los ejemplares.
- RN-10. El máximo de préstamos simultáneos es parametrizable.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Lector, Autor, Editorial, Libro, LibroAutor, Ejemplar, Prestamo, Renovacion, Reserva, Sancion, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar usuarios y roles.
- RF-02 Registrar autores y editoriales.
- RF-03 Gestionar títulos bibliográficos.
- RF-04 Registrar ejemplares con código único.
- RF-05 Buscar catálogo por título, autor, ISBN o categoría.
- RF-06 Consultar disponibilidad por título.
- RF-07 Registrar préstamo de ejemplar.
- RF-08 Validar límite de préstamos y sanciones.
- RF-09 Registrar devolución.
- RF-10 Calcular mora operativa.
- RF-11 Registrar reserva sobre un título.
- RF-12 Asignar prioridad de reserva.
- RF-13 Renovar préstamo si cumple reglas.
- RF-14 Liberar reservas vencidas.
- RF-15 Consultar historial del lector.
- RF-16 Consultar historial del ejemplar.
- RF-17 Mostrar tablero de préstamos, mora y disponibilidad.
- RF-18 Auditar operaciones críticas.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Dashboard
- Catálogo administrativo
- Gestión de ejemplares
- Préstamo/devolución en mostrador
- Reservas
- Usuarios lectores
- Sanciones
- Reportes operativos

- Auditoría

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Explorar catálogo
- Buscar y filtrar
- Detalle y disponibilidad
- Crear/cancelar reserva
- Mis préstamos
- Fechas de devolución
- Historial personal
- Avisos internos

J. Flujo crítico que debe demostrarse extremo a extremo

Bibliotecario registra título y ejemplares → lector consulta disponibilidad y reserva → bibliotecario entrega un ejemplar → se crea préstamo → lector consulta vencimiento en móvil → bibliotecario registra devolución → sistema libera o asigna según cola de reservas.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- POST /auth/login
- GET/POST /books
- GET/POST /copies
- GET /catalog/search
- POST /loans
- POST /loans/{id}/return
- POST /loans/{id}/renew
- POST /reservations
- DELETE /reservations/{id}
- GET /me/loans
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.
- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede generar resúmenes de novedades del catálogo o responder preguntas sobre disponibilidad usando exclusivamente datos recuperados por herramientas internas. No debe inventar títulos ni estados.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.
- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.

- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- Diferencie Libro y Ejemplar en su modelo.
- ¿Cómo evita dos préstamos simultáneos del mismo ejemplar?
- Explique la cola de reservas.
- ¿Qué validaciones ocurren al renovar?
- Muestre cómo la app móvil obtiene “Mis préstamos”.
- ¿Qué índices usaría para búsqueda frecuente?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

13. CargoTrack - Paquetería, Rutas, Estados y Prueba de Entrega

A. Situación del cliente

Una empresa local de mensajería necesita administrar envíos desde su recepción hasta la entrega. El cliente requiere conocer remitente, destinatario, paquete, ruta, mensajero, estados, incidencias y prueba de entrega, evitando actualizaciones de estado incoherentes.

B. Objetivo contractual del producto

Implementar una solución que permita registrar envíos, asignarlos a rutas/mensajeros, controlar una máquina de estados, registrar incidencias y confirmar entregas desde móvil con evidencia mínima.

C. Actores y responsabilidades

Actor	Responsabilidad
Operador	Registra envíos y datos de remitente/destinatario.
Coordinador	Agrupar envíos, crea rutas y asigna mensajeros.
Mensajero	Consulta ruta, actualiza estados y registra entrega/incidencia.
Administrador	Configura usuarios y parámetros.
Cliente consultante	Consulta seguimiento mediante código público.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Usuarios y roles
- Clientes/contactos
- Envíos
- Paquetes
- Rutas
- Asignaciones
- Seguimiento de estados
- Incidencias
- Prueba de entrega
- Consulta pública
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. Cada envío tiene código de seguimiento único.
- RN-02. Los estados sólo pueden avanzar según transiciones permitidas.
- RN-03. Un envío entregado no puede volver a “en ruta”.
- RN-04. Sólo el mensajero asignado puede actualizar ciertos estados operativos.

- RN-05. Una ruta sólo puede cerrarse cuando todos sus envíos tienen estado terminal o incidencia justificada.
- RN-06. La entrega requiere nombre de receptor y marca temporal; la fotografía es opcional según configuración académica.
- RN-07. Una incidencia debe tener tipo, descripción y responsable.
- RN-08. Cambios de dirección después de despacho requieren autorización.
- RN-09. El historial de estados es inmutable.
- RN-10. La consulta pública no expone datos personales sensibles.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Contacto, Envío, Paquete, Ruta, RutaEnvío, AsignacionMensajero, HistorialEstado, Incidencia, PruebaEntrega, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Registrar envíos.
- RF-02 Generar código único de seguimiento.
- RF-03 Gestionar datos mínimos de remitente y destinatario.
- RF-04 Registrar uno o más paquetes por envío.
- RF-05 Crear rutas de reparto.
- RF-06 Asignar mensajero a ruta.
- RF-07 Incorporar envíos a una ruta.
- RF-08 Cambiar estado validando transición.
- RF-09 Consultar historial cronológico.
- RF-10 Registrar salida a reparto.
- RF-11 Confirmar entrega.
- RF-12 Registrar incidencia.
- RF-13 Reprogramar envío con autorización.
- RF-14 Consultar seguimiento público por código.
- RF-15 Mostrar ruta del mensajero en móvil.
- RF-16 Filtrar pendientes por estado/zona.
- RF-17 Mostrar KPIs de entregas/incidencias.
- RF-18 Auditar cambios sensibles.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Registro de envíos
- Dashboard
- Planificación de rutas
- Asignación de mensajeros
- Seguimiento de envíos

- Gestión de incidencias
- Consulta de prueba de entrega
- Búsqueda por código
- Administración

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Mi ruta del día
- Lista ordenada de entregas
- Detalle del envío
- Actualizar estado
- Registrar incidencia
- Confirmar receptor/entrega
- Historial reciente
- Modo de reintento ante falla de red básica

J. Flujo crítico que debe demostrarse extremo a extremo

Operador registra envío → coordinador lo asigna a ruta y mensajero → mensajero inicia ruta en móvil → actualiza “en reparto” → registra entrega o incidencia → historial queda trazado → cliente consulta estado mediante código.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- POST /shipments
- GET /shipments/{tracking}
- POST /routes
- POST /routes/{id}/shipments
- POST /routes/{id}/assign
- GET /me/routes/current
- POST /shipments/{id}/status
- POST /shipments/{id}/incidents
- POST /shipments/{id}/delivery-proof
- GET /tracking/{code}
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.

- La documentación coincide con el sistema entregado.
- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede resumir incidencias recurrentes o producir una explicación operativa del día a partir de datos del sistema. No puede cambiar estados ni decidir entregas automáticamente.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.

- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.
- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.
- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- Modele las transiciones de estado y explique dónde se validan.
- ¿Cómo garantiza que el historial no se sobrescriba?
- ¿Qué datos ocultaría en la consulta pública?
- ¿Cómo maneja una actualización móvil duplicada?
- Muestre la relación Ruta-Envío.
- ¿Qué pasa si una ruta intenta cerrarse con pendientes?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

14. SalonPro - Agenda, Servicios, Profesionales y Operación de Salón/SPA

A. Situación del cliente

Un salón de belleza y spa necesita coordinar profesionales, servicios, horarios, citas, atención, consumos internos y pagos simplificados. El principal problema actual son dobles reservas y desconocimiento de disponibilidad real.

B. Objetivo contractual del producto

Crear un MVP que permita configurar servicios y profesionales, gestionar disponibilidad, reservar citas desde web/móvil, controlar estados de atención y obtener indicadores básicos de ocupación.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Configura servicios, profesionales, horarios y parámetros.
Recepcionista	Agenda, reprograma y cancela citas.
Profesional	Consulta agenda y actualiza atención.
Cliente	Reserva, consulta y cancela según reglas.
Supervisor	Consulta KPIs y excepciones.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Usuarios
- Profesionales
- Servicios
- Horarios
- Disponibilidad
- Clientes
- Citas
- Atención
- Pagos simplificados
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. No puede existir solapamiento de citas para un mismo profesional.
- RN-02. La duración de una cita depende del servicio seleccionado.
- RN-03. Sólo profesionales habilitados para un servicio pueden ser asignados.
- RN-04. Las cancelaciones tardías quedan registradas.

- RN-05. Una cita confirmada debe recorrer estados válidos: reservada, confirmada, en atención, finalizada/cancelada/no asistió.
- RN-06. No se eliminan citas históricas.
- RN-07. Los horarios bloqueados no están disponibles para reserva.
- RN-08. El precio aplicado queda congelado al confirmar la cita.
- RN-09. Un cliente puede tener varias citas, pero se controla solapamiento personal configurable.
- RN-10. El pago del MVP es registro operativo, no integración bancaria real.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Profesional, Servicio, ProfesionalServicio, Horario, BloqueoHorario, Cliente, Cita, CitaServicio, Atencion, Pago, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar profesionales.
- RF-02 Gestionar servicios y duración.
- RF-03 Asociar profesionales a servicios.
- RF-04 Configurar horarios.
- RF-05 Registrar bloqueos de agenda.
- RF-06 Consultar slots disponibles.
- RF-07 Registrar clientes.
- RF-08 Crear cita.
- RF-09 Reprogramar cita.
- RF-10 Cancelar cita con motivo.
- RF-11 Confirmar asistencia.
- RF-12 Iniciar y finalizar atención.
- RF-13 Registrar no asistencia.
- RF-14 Registrar pago simplificado.
- RF-15 Consultar agenda diaria/semanal.
- RF-16 Consultar historial del cliente.
- RF-17 Mostrar ocupación e ingresos operativos.
- RF-18 Auditar cambios de agenda.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Dashboard
- Calendario de agenda
- CRUD de servicios
- CRUD de profesionales
- Horarios/bloqueos
- Clientes

- Detalle de cita
- Atención y pago
- Indicadores

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Mi agenda (profesional)
- Buscar disponibilidad (cliente)
- Crear cita
- Mis citas
- Cancelar/reprogramar
- Actualizar estado de atención
- Resumen diario

J. Flujo crítico que debe demostrarse extremo a extremo

Administrador configura servicio/profesional/horario → cliente consulta disponibilidad y reserva → recepcionista confirma → profesional inicia y finaliza atención → se registra pago → dashboard refleja ocupación.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- GET/POST /services
- GET/POST /professionals
- POST /availability/search
- POST /appointments
- PATCH /appointments/{id}/reschedule
- PATCH /appointments/{id}/cancel
- PATCH /appointments/{id}/status
- POST /appointments/{id}/payment
- GET /me/appointments
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.
- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede redactar resúmenes de agenda o sugerir mensajes de recordatorio a partir de citas existentes. No puede modificar reservas por sí mismo.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.
- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.

- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- Explique cómo detecta solapamientos.
- ¿Cómo calcula los slots disponibles?
- ¿Dónde congela el precio de la cita?
- ¿Qué diferencia existe entre horario y bloqueo?
- ¿Cómo se sincroniza móvil y backend al reprogramar?
- ¿Qué restricciones pondría a cancelaciones?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

15. EquipRent - Alquiler de Equipos, Reservas, Entregas, Devoluciones y Daños

A. Situación del cliente

Una empresa alquila herramientas, equipos audiovisuales y maquinaria ligera. Requiere controlar unidades físicas, disponibilidad por fechas, reservas, contratos operativos, entregas, devoluciones, depósitos y daños, evitando que la misma unidad sea comprometida a dos clientes.

B. Objetivo contractual del producto

Construir un sistema que gestione catálogo y unidades, disponibilidad temporal, reservas, entregas y devoluciones con inspección, manteniendo trazabilidad completa de cada activo.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Configura catálogo, tarifas, usuarios y políticas.
Operador	Registra clientes, reservas, entregas y devoluciones.
Técnico	Inspecciona unidades y registra daños/mantenimiento.
Cliente	Consulta reservas y estado desde móvil.
Supervisor	Autoriza excepciones y consulta indicadores.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Clientes
- Catálogo
- Unidades físicas
- Tarifas
- Disponibilidad
- Reservas
- Entrega
- Devolución
- Inspección/daños
- Mantenimiento
- Depósitos
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. Cada unidad física tiene identificador único y estado.
- RN-02. Una unidad no puede asignarse a reservas solapadas.

- RN-03. La disponibilidad se calcula por rango de fechas.
- RN-04. La reserva no equivale a entrega física.
- RN-05. La entrega requiere reserva confirmada o autorización excepcional.
- RN-06. La devolución registra condición de la unidad.
- RN-07. Un daño puede bloquear la unidad para nuevas reservas.
- RN-08. Los depósitos se registran como información operativa; no hay pasarela bancaria real.
- RN-09. Una reserva cancelada libera disponibilidad.
- RN-10. El historial del activo no se elimina.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Cliente, CategoriaEquipo, Equipo, UnidadEquipo, Tarifa, Reserva, ReservaDetalle, Entrega, Devolucion, Inspeccion, Dano, Mantenimiento, Deposito, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar catálogo de equipos.
- RF-02 Registrar unidades físicas.
- RF-03 Configurar tarifas.
- RF-04 Registrar clientes.
- RF-05 Consultar disponibilidad por rango.
- RF-06 Crear reserva con varias unidades/tipos.
- RF-07 Validar conflictos de fechas.
- RF-08 Confirmar/cancelar reserva.
- RF-09 Registrar entrega física.
- RF-10 Registrar depósito operativo.
- RF-11 Registrar devolución.
- RF-12 Inspeccionar condición.
- RF-13 Registrar daño.
- RF-14 Bloquear unidad por mantenimiento.
- RF-15 Consultar historial de unidad.
- RF-16 Consultar reservas del cliente.
- RF-17 Mostrar utilización y disponibilidad.
- RF-18 Auditar excepciones.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Dashboard
- Catálogo/unidades
- Calendario de disponibilidad
- Clientes
- Reservas

- Entrega
- Devolución e inspección
- Daños/mantenimiento
- Indicadores

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Consulta de catálogo
- Disponibilidad
- Mis reservas
- Detalle de retiro/devolución
- Checklist de inspección para técnico
- Registro de daño con descripción
- Estado de unidad

J. Flujo crítico que debe demostrarse extremo a extremo

Operador registra cliente → consulta disponibilidad por fechas → reserva unidades → en retiro registra entrega → cliente consulta reserva en móvil → al retorno técnico inspecciona → registra devolución y posible daño → unidad vuelve a disponible o mantenimiento.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- GET/POST /equipment
- GET/POST /units
- POST /availability/search
- POST /reservations
- PATCH /reservations/{id}/confirm
- POST /reservations/{id}/delivery
- POST /reservations/{id}/return
- POST /units/{id}/damage
- POST /units/{id}/maintenance
- GET /me/reservations
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.

- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede resumir observaciones de inspección o agrupar causas frecuentes de daño. La decisión de cobrar, bloquear o liberar una unidad sigue siendo una regla determinista/autorizada.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.

- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.
- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- ¿Cómo evita doble reserva por rango de fechas?
- Diferencie Equipo y UnidadEquipo.
- ¿Qué transacción usaría en una entrega?
- ¿Cómo un daño afecta disponibilidad futura?
- Muestre el historial completo de una unidad.
- ¿Qué datos consume la pantalla móvil de reservas?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

16. AgroControl - Gestión de Lotes Agrícolas, Campañas, Labores, Insumos y Cosecha

A. Situación del cliente

Una unidad productiva agrícola administra parcelas, campañas, cultivos, labores de campo, uso de insumos, responsables y cosechas con registros manuales. El cliente requiere trazabilidad operativa por lote agrícola y campaña, sin pretender sustituir asesoría agronómica profesional.

B. Objetivo contractual del producto

Desarrollar un sistema web/móvil para planificar campañas, registrar labores e insumos, controlar responsables y capturar cosechas, ofreciendo una bitácora completa por parcela.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Configura predios, usuarios y catálogos.
Jefe de campo	Planifica campañas y labores.
Operario	Consulta y reporta labores desde móvil.
Almacenero	Registra salidas de insumos para labores.
Supervisor	Consulta avance e indicadores.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Predios y parcelas
- Cultivos
- Campañas
- Labores
- Asignaciones
- Insumos
- Movimientos de insumos
- Bitácora de campo
- Cosecha
- Incidencias
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. Una parcela puede participar en campañas distintas en el tiempo, no superpuestas si la regla de ocupación lo impide.
- RN-02. Toda labor pertenece a una campaña y parcela.

- RN-03. Una labor tiene estado planificada, asignada, en ejecución, completada o cancelada.
- RN-04. El consumo de insumos no puede superar stock disponible.
- RN-05. Los operarios sólo reportan labores asignadas.
- RN-06. La cosecha se registra contra una campaña activa/finalizable.
- RN-07. Las cantidades y unidades de medida deben estar explícitas.
- RN-08. Las incidencias quedan vinculadas a parcela/campaña/labor.
- RN-09. No se elimina historial de campañas terminadas.
- RN-10. La IA no prescribe dosis ni tratamientos agronómicos.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Predio, Parcela, Cultivo, Campaña, Labor, AsignacionLabor, Insumo, MovimientoInsumo, ConsumoLabor, BitacoraCampo, Incidencia, Cosecha, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar predios y parcelas.
- RF-02 Gestionar cultivos.
- RF-03 Crear campañas.
- RF-04 Planificar labores.
- RF-05 Asignar operarios.
- RF-06 Consultar agenda de campo.
- RF-07 Iniciar/completar labor.
- RF-08 Registrar observación de campo.
- RF-09 Gestionar insumos.
- RF-10 Registrar entradas/salidas de insumos.
- RF-11 Asociar consumo a labor.
- RF-12 Validar stock antes de consumo.
- RF-13 Registrar incidencias.
- RF-14 Registrar cosecha.
- RF-15 Consultar bitácora por parcela.
- RF-16 Consultar avance por campaña.
- RF-17 Mostrar KPIs operativos.
- RF-18 Auditar cambios críticos.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Mapa/listado de parcelas
- Campañas
- Planificador de labores
- Insumos
- Asignaciones

- Bitácora
- Cosechas
- Incidencias
- Dashboard

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Mis labores
- Detalle de parcela/campaña
- Iniciar/finalizar labor
- Registrar consumo
- Registrar observación/incidencia
- Capturar cosecha
- Historial reciente

J. Flujo crítico que debe demostrarse extremo a extremo

Jefe crea campaña sobre parcela → planifica labores → asigna operario → almacén entrega insumo → operario ejecuta y reporta desde móvil → se actualiza bitácora → se registra cosecha → supervisor revisa indicadores.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- GET/POST /plots
- GET/POST /campaigns
- GET/POST /tasks
- POST /tasks/{id}/assign
- PATCH /tasks/{id}/status
- POST /tasks/{id}/inputs
- POST /tasks/{id}/notes
- POST /campaigns/{id}/harvests
- GET /plots/{id}/logbook
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.
- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede resumir la bitácora de una campaña o clasificar textualmente incidencias ya registradas. No debe generar recomendaciones técnicas de fertilización o plaguicidas como verdad operativa.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.
- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.

- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- Explique la relación parcela–campana–labor.
- ¿Cómo asegura consistencia de unidades de medida?
- ¿Qué pasa si se intenta consumir más insumo del disponible?
- ¿Cómo construye la bitácora?
- ¿Qué puede registrar el operario móvil?
- ¿Dónde termina la responsabilidad de Spring AI?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

17. BuildTrack - Seguimiento de Obra, Tareas, Materiales, Avances e Incidencias

A. Situación del cliente

Una empresa constructora pequeña necesita controlar proyectos de obra, frentes de trabajo, tareas, responsables, solicitudes de materiales, entregas, avances diarios e incidencias. El cliente requiere una visión operativa del progreso; no se pretende reemplazar herramientas de cálculo estructural ni gestión financiera formal.

B. Objetivo contractual del producto

Crear una plataforma colaborativa web/móvil que registre planificación de tareas y avances, materiales solicitados/entregados e incidencias, con trazabilidad por proyecto y responsable.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Gestiona usuarios y catálogos.
Jefe de obra	Planifica frentes, tareas y revisa avances.
Encargado de almacén	Atiende solicitudes de material.
Capataz/Operario	Reporta avance e incidencias desde móvil.
Supervisor	Consulta cumplimiento y desviaciones.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Proyectos
- Frentes de trabajo
- Tareas
- Asignaciones
- Partes diarios
- Materiales
- Solicitudes
- Entregas
- Incidencias
- Evidencias
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. Toda tarea pertenece a un frente y proyecto.
- RN-02. Una tarea tiene responsable, fechas planificadas y estado.

- RN-03. El porcentaje de avance debe estar entre 0 y 100 y no disminuir sin autorización.
- RN-04. Una solicitud de material debe tener estado y detalle.
- RN-05. No se puede entregar más material que lo aprobado/solicitado sin autorización.
- RN-06. Un parte diario es inmutable después de cierre salvo corrección auditada.
- RN-07. Una incidencia puede bloquear una tarea.
- RN-08. Los archivos/fotos son evidencia complementaria; el MVP puede almacenar referencia de archivo.
- RN-09. Los históricos no se eliminan.
- RN-10. El sistema no calcula estructuras ni certificaciones oficiales.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Proyecto, FrenteTrabajo, Tarea, Asignacion, ParteDiario, Material, SolicitudMaterial, SolicitudDetalle, EntregaMaterial, Incidencia, Evidencia, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar proyectos.
- RF-02 Crear frentes de trabajo.
- RF-03 Crear tareas con fechas y responsable.
- RF-04 Actualizar estado y porcentaje de avance.
- RF-05 Registrar parte diario.
- RF-06 Registrar observaciones/evidencias.
- RF-07 Gestionar catálogo de materiales.
- RF-08 Crear solicitud de material.
- RF-09 Aprobar/rechazar solicitud.
- RF-10 Registrar entrega de material.
- RF-11 Validar cantidades.
- RF-12 Registrar incidencia.
- RF-13 Marcar incidencia como bloqueante.
- RF-14 Consultar cronología del proyecto.
- RF-15 Consultar tareas por responsable.
- RF-16 Consultar consumo/entrega por material.
- RF-17 Mostrar KPIs de avance y pendientes.
- RF-18 Auditar cambios excepcionales.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Dashboard de proyecto
- Frentes y tareas
- Plan de trabajo
- Solicitudes de material
- Almacén/entregas

- Partes diarios
- Incidencias
- Cronología
- Reportes operativos

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Mis tareas
- Registrar avance
- Parte diario rápido
- Solicitar material
- Registrar incidencia
- Adjuntar referencia de evidencia
- Consultar estado de solicitudes

J. Flujo crítico que debe demostrarse extremo a extremo

Jefe crea proyecto/frente/tarea → asigna responsable → capataz reporta avance desde móvil → solicita material → almacén aprueba/entrega → se registra parte diario e incidencia si existe → dashboard consolida estado del proyecto.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- GET/POST /projects
- GET/POST /projects/{id}/workfronts
- GET/POST /tasks
- PATCH /tasks/{id}/progress
- POST /daily-reports
- POST /material-requests
- PATCH /material-requests/{id}/approve
- POST /material-requests/{id}/deliveries
- POST /incidents
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.

- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede sintetizar partes diarios e incidencias en un resumen ejecutivo basado en datos ya registrados. No debe emitir cálculos de ingeniería estructural ni decisiones de seguridad como autoridad.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.

- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.
- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- ¿Cómo modela avance sin perder historial?
- ¿Qué diferencia hay entre solicitud y entrega?
- ¿Cómo valida una entrega parcial?
- ¿Cómo representa una incidencia bloqueante?
- ¿Qué pantalla móvil usaría el capataz?
- ¿Qué datos alimentan el dashboard?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

18. CampusLab - Reservas de Laboratorios, Equipamiento, Prácticas y Control de Uso

A. Situación del cliente

Una facultad necesita administrar laboratorios, horarios, equipamiento, reservas de docentes, prácticas planificadas, asistencia básica y reportes de incidencias. Actualmente se producen cruces de horario y no existe una bitácora clara de uso de equipos.

B. Objetivo contractual del producto

Implementar una plataforma web/móvil para reservar laboratorios y recursos, validar conflictos, controlar sesiones/prácticas y registrar incidencias de equipos.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Configura laboratorios, equipos y usuarios.
Docente	Solicita reservas y planifica prácticas.
Encargado de laboratorio	Aprueba, abre/cierra sesiones y registra incidencias.
Estudiante	Consulta prácticas y asistencia propia.
Supervisor académico	Consulta utilización e incidencias.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Usuarios y roles
- Laboratorios
- Equipamiento
- Disponibilidad
- Reservas
- Prácticas
- Sesiones
- Asistencia básica
- Incidencias
- Mantenimiento de equipo
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. No pueden existir reservas solapadas para el mismo laboratorio.
- RN-02. Una reserva puede requerir aprobación según tipo.

- RN-03. El aforo del laboratorio limita participantes declarados.
- RN-04. Un equipo fuera de servicio no puede asignarse como recurso disponible.
- RN-05. Una práctica debe vincularse a una reserva confirmada.
- RN-06. Una sesión registra apertura y cierre real.
- RN-07. Una incidencia puede sacar un equipo de servicio.
- RN-08. La asistencia es académica-operativa básica, no reemplaza el sistema oficial institucional.
- RN-09. Las reservas canceladas liberan el horario.
- RN-10. Los históricos de uso no se eliminan.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Laboratorio, Equipo, Disponibilidad, Reserva, ReservaRecurso, Practica, SesionLaboratorio, Participante, Asistencia, IncidenciaEquipo, Mantenimiento, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar laboratorios.
- RF-02 Gestionar equipos por laboratorio.
- RF-03 Configurar disponibilidad.
- RF-04 Consultar calendario.
- RF-05 Solicitar reserva.
- RF-06 Validar solapamiento y aforo.
- RF-07 Aprobar/rechazar reserva.
- RF-08 Cancelar reserva.
- RF-09 Registrar práctica.
- RF-10 Abrir sesión de laboratorio.
- RF-11 Cerrar sesión.
- RF-12 Registrar participantes/asistencia básica.
- RF-13 Registrar incidencia de equipo.
- RF-14 Cambiar equipo a fuera de servicio.
- RF-15 Registrar mantenimiento.
- RF-16 Consultar mis prácticas en móvil.
- RF-17 Mostrar utilización por laboratorio.
- RF-18 Auditar cambios de reserva.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Calendario de laboratorios
- Reservas
- Aprobaciones
- Catálogo de equipos
- Prácticas

- Sesiones
- Asistencia
- Incidencias/mantenimiento
- Dashboard

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Mis prácticas
- Detalle de reserva
- Check-in académico simple
- Incidencias rápidas
- Estado de equipos
- Agenda del encargado
- Cerrar sesión

J. Flujo crítico que debe demostrarse extremo a extremo

Docente consulta disponibilidad → solicita laboratorio/equipos → encargado aprueba → se crea práctica → estudiante consulta en móvil → encargado abre sesión → registra asistencia/incidencias → cierra sesión → dashboard actualiza utilización.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- GET/POST /labs
- GET/POST /equipment
- POST /availability/search
- POST /reservations
- PATCH /reservations/{id}/approve
- POST /practices
- POST /sessions/{id}/open
- POST /sessions/{id}/attendance
- POST /equipment/{id}/incidents
- POST /sessions/{id}/close
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.

- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede resumir incidencias por laboratorio o generar una explicación de patrones de utilización. No toma decisiones automáticas de aprobación.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.

- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.
- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- ¿Cómo evita cruces de reserva?
- ¿Cómo incorpora el aforo en la validación?
- Diferencie Reserva, Práctica y Sesión.
- ¿Qué ocurre cuando un equipo queda fuera de servicio?
- ¿Cómo implementa asistencia básica sin mezclarla con IAM?
- ¿Qué ve un estudiante en móvil?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

19. ONGImpact - Voluntariado, Campañas Sociales, Actividades, Beneficiarios e Impacto

A. Situación del cliente

Una organización social coordina campañas, voluntarios, actividades y atención a beneficiarios. Necesita ordenar inscripciones, asignaciones, asistencia, entrega de ayudas e indicadores, cuidando especialmente la privacidad y evitando exposición innecesaria de datos sensibles.

B. Objetivo contractual del producto

Construir un sistema que gestione campañas y actividades, registro de voluntarios, asignaciones y participación, además de beneficiarios con información mínima y trazabilidad de entregas/atenciones.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Gestiona usuarios, catálogos y parámetros.
Coordinador	Crea campañas, actividades y asignaciones.
Voluntario	Consulta y confirma sus actividades en móvil.
Operador social	Registra beneficiarios mínimos y entregas.
Supervisor	Consulta cobertura e indicadores.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Voluntarios
- Campañas
- Actividades
- Inscripciones
- Asignaciones
- Asistencia
- Beneficiarios mínimos
- Atenciones/entregas
- Inventario básico de ayuda
- Consentimiento/privacidad básica
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. Se recopilan sólo datos necesarios de beneficiarios.
- RN-02. El acceso a datos de beneficiarios está restringido por rol.
- RN-03. Un voluntario no puede ser asignado a actividades con horarios solapados.

- RN-04. Cada actividad tiene cupo y estado.
- RN-05. La asistencia se registra sólo para participantes asignados/inscritos.
- RN-06. Una entrega a beneficiario debe tener tipo, cantidad, fecha y responsable.
- RN-07. No se exponen datos personales en tableros agregados.
- RN-08. Las cancelaciones quedan registradas.
- RN-09. El historial de campañas se conserva.
- RN-10. Spring AI no debe recibir datos sensibles innecesarios.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Voluntario, Campana, Actividad, Inscripcion, Asignacion, Asistencia, Beneficiario, Consentimiento, TipoAyuda, InventarioAyuda, EntregaAyuda, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar perfil de voluntario.
- RF-02 Crear campañas.
- RF-03 Crear actividades con cupo/horario.
- RF-04 Inscribir voluntarios.
- RF-05 Validar cupo y solapamiento.
- RF-06 Asignar responsables.
- RF-07 Confirmar participación.
- RF-08 Registrar asistencia.
- RF-09 Registrar beneficiario con datos mínimos.
- RF-10 Registrar consentimiento básico cuando aplique.
- RF-11 Gestionar tipos de ayuda.
- RF-12 Registrar stock de ayuda básico.
- RF-13 Registrar entrega a beneficiario.
- RF-14 Consultar historial de actividad.
- RF-15 Consultar mis actividades en móvil.
- RF-16 Consultar cobertura agregada.
- RF-17 Mostrar KPIs sin exponer datos sensibles.
- RF-18 Auditar accesos/operaciones críticas.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Campañas
- Actividades
- Voluntarios
- Asignaciones
- Asistencia
- Beneficiarios

- Entregas/ayudas
- Indicadores agregados
- Auditoría

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Mis actividades
- Inscribirme/retirarme
- Confirmar participación
- Check-in simple
- Detalle logístico
- Registrar entrega autorizada
- Notificaciones internas

J. Flujo crítico que debe demostrarse extremo a extremo

Coordinador crea campaña/actividad → voluntario se inscribe → sistema valida cupo/horario → coordinador confirma asignación → voluntario consulta actividad en móvil → se registra asistencia → operador registra entregas → dashboard muestra impacto agregado.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- GET/POST /campaigns
- GET/POST /activities
- POST /activities/{id}/registrations
- POST /activities/{id}/assignments
- POST /activities/{id}/attendance
- GET/POST /beneficiaries
- POST /aid/deliveries
- GET /me/activities
- GET /impact/dashboard
- GET /audit

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.
- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede generar resúmenes narrativos de impacto usando exclusivamente datos agregados y anonimizados. No debe recibir ni reproducir información personal sensible de beneficiarios.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.
- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.

- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- ¿Qué datos mínimos guarda de un beneficiario y por qué?
- ¿Cómo evita solapamientos de voluntarios?
- ¿Cómo protege el dashboard contra exposición de PII?
- ¿Cómo modela una entrega de ayuda?
- ¿Qué controles aplica antes de enviar contexto al LLM?
- ¿Qué diferencia existe entre inscripción y asignación?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

20. InmoRent - Inmuebles, Contratos de Alquiler, Cuotas, Incidencias y Mantenimiento

A. Situación del cliente

Una empresa administradora de propiedades necesita controlar inmuebles, unidades, arrendatarios, contratos, cuotas operativas, pagos registrados, incidencias y solicitudes de mantenimiento. El cliente no necesita contabilidad general ni pasarela bancaria; requiere gestión operacional y trazabilidad.

B. Objetivo contractual del producto

Crear un MVP web/móvil para administrar ocupación, contratos, calendario de cuotas, registro de pagos, incidencias del arrendatario y órdenes básicas de mantenimiento.

C. Actores y responsabilidades

Actor	Responsabilidad
Administrador	Gestiona inmuebles, unidades, usuarios y parámetros.
Gestor inmobiliario	Registra contratos, cuotas y seguimiento.
Arrendatario	Consulta contrato, cuotas e incidencias desde móvil.
Técnico de mantenimiento	Consulta y actualiza órdenes asignadas.
Supervisor	Consulta ocupación, mora e incidencias.

D. Alcance funcional cerrado

El MVP obligatorio comprende los siguientes módulos. El equipo puede mejorar UX o agregar facilidades, pero no puede eliminar estos componentes sin autorización docente:

- Inmuebles y unidades
- Arrendatarios
- Contratos
- Cuotas
- Pagos operativos
- Incidencias
- Órdenes de mantenimiento
- Asignaciones técnicas
- Documentos referenciales
- Dashboard

E. Reglas de negocio obligatorias

- RN-01. Una unidad no puede tener dos contratos activos con periodos incompatibles.
- RN-02. Todo contrato tiene fecha de inicio, fin y estado.
- RN-03. Las cuotas se generan según periodicidad definida.

- RN-04. Un pago se aplica a una o más cuotas según reglas del MVP.
- RN-05. Los importes históricos del contrato no se sobrescriben.
- RN-06. Una incidencia pertenece a una unidad/contrato activo cuando corresponde.
- RN-07. Una incidencia puede generar una orden de mantenimiento.
- RN-08. El técnico sólo actualiza órdenes asignadas.
- RN-09. El sistema registra pagos pero no realiza conciliación bancaria ni contabilidad oficial.
- RN-10. Los contratos e históricos no se eliminan físicamente.

F. Modelo de información mínimo esperado

Como mínimo, el equipo deberá modelar las siguientes entidades/conceptos. Puede refinar agregados y nombres si justifica la decisión y preserva el significado del dominio:

Usuario, Rol, Inmueble, Unidad, Arrendatario, Contrato, Cuota, Pago, PagoAplicacion, Incidencia, OrdenMantenimiento, AsignacionTecnico, DocumentoReferencia, Auditoria.

El modelo relacional final debe incluir claves, restricciones, cardinalidades, estados, auditoría mínima e índices justificados para consultas relevantes. No se acepta una tabla genérica de “datos” que evite modelar el dominio.

G. Requisitos funcionales predefinidos

- RF-01 Gestionar inmuebles.
- RF-02 Gestionar unidades y estado de ocupación.
- RF-03 Registrar arrendatarios.
- RF-04 Crear contrato.
- RF-05 Validar solapamiento de contratos.
- RF-06 Generar calendario de cuotas.
- RF-07 Consultar cuotas pendientes/pagadas.
- RF-08 Registrar pago operativo.
- RF-09 Aplicar pago a cuota.
- RF-10 Registrar incidencia.
- RF-11 Clasificar prioridad de incidencia.
- RF-12 Crear orden de mantenimiento.
- RF-13 Asignar técnico.
- RF-14 Actualizar estados de orden.
- RF-15 Consultar mis cuotas en móvil.
- RF-16 Consultar mis incidencias.
- RF-17 Mostrar ocupación, mora e incidencias.
- RF-18 Auditar operaciones críticas.

H. Requisitos de experiencia web

La aplicación web se orienta principalmente a roles administrativos y operativos. Debe ser funcional, navegable y conectada a la API real. Como mínimo:

- Dashboard
- Inmuebles/unidades
- Arrendatarios
- Contratos
- Cuotas
- Registro de pagos

- Incidencias
- Órdenes de mantenimiento
- Auditoría

I. Requisitos de experiencia móvil

La aplicación móvil no es una réplica completa del web: debe priorizar tareas de campo o consulta inmediata. Como mínimo:

- Mi contrato
- Mis cuotas
- Historial de pagos
- Crear incidencia
- Seguimiento de incidencia
- Órdenes asignadas para técnico
- Actualizar estado de trabajo

J. Flujo crítico que debe demostrarse extremo a extremo

Gestor registra unidad/arrendatario → crea contrato → sistema genera cuotas → arrendatario consulta móvil → gestor registra pago → arrendatario reporta incidencia → se crea/asigna orden → técnico actualiza estado → dashboard refleja ocupación y pendientes.

K. API REST mínima esperada

Los paths son una referencia contractual de cobertura; el equipo puede refinarlos manteniendo semántica REST y trazabilidad:

- GET/POST /properties
- GET/POST /units
- GET/POST /tenants
- POST /contracts
- POST /contracts/{id}/installments/generate
- POST /payments
- POST /incidents
- POST /maintenance-orders
- PATCH /maintenance-orders/{id}/assign
- PATCH /maintenance-orders/{id}/status
- GET /me/installments
- GET /dashboard

L. Criterios de aceptación global del proyecto

- El sistema puede inicializarse desde cero mediante instrucciones reproducibles y migraciones.
- El flujo crítico se ejecuta sin editar manualmente tablas durante la demostración.
- Las reglas de negocio obligatorias se validan en backend, no únicamente en la interfaz.
- Web y móvil consumen datos reales del mismo backend.
- Los errores relevantes producen respuestas controladas y mensajes comprensibles.
- Las operaciones críticas conservan trazabilidad suficiente para explicar quién hizo qué y cuándo.
- Los estados y transiciones del dominio no permiten combinaciones incoherentes.
- Existen pruebas automatizadas sobre reglas y flujos esenciales.
- La documentación coincide con el sistema entregado.

- Cualquier integrante puede defender arquitectura, datos, backend, web, móvil y pruebas del alcance exigido.

M. Uso acotado de Spring AI

Spring AI puede resumir incidencias de mantenimiento o generar un resumen operativo del portafolio a partir de datos calculados. No interpreta contratos como asesor legal ni determina obligaciones jurídicas.

El equipo debe documentar prompt/contexto, datos enviados al modelo, manejo de errores, límites y por qué la funcionalidad sigue siendo segura si el proveedor de IA no responde.

N. Entregas del proyecto

PARTE I - Parcial 1: dominio, requisitos, datos y backend fundacional

- Documento de comprensión del cliente: problema, actores, alcance, exclusiones y glosario específico del dominio.
- Matriz completa de requisitos funcionales y reglas de negocio del proyecto, indicando cuáles entran en el corte.
- Casos de uso/historias del primer flujo y criterios de aceptación Given-When-Then.
- Modelo conceptual y modelo relacional normalizado; PK, FK, nulabilidad, restricciones e índices iniciales.
- Migraciones PostgreSQL para el núcleo del dominio y datos semilla mínimos.
- Estructura del backend Java 21/Spring Boot en módulos y puertos/adaptadores; entidades/VO y primeros casos de uso.
- API inicial ejecutable para catálogos y al menos una operación de dominio relevante.
- React + TypeScript: shell de aplicación, rutas principales y al menos una pantalla que consuma API real.
- React Native + TypeScript: navegación base y al menos una pantalla conectada a un endpoint real.
- README de ejecución local, estrategia Git y evidencia de PRs.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE II - Trabajos Prácticos Integradores: primer flujo extremo a extremo

- Actualización de trazabilidad requisito → regla → caso de uso → endpoint → pantalla → prueba.
- Persistencia completa del primer flujo transaccional con manejo de errores y validaciones.
- Implementación de reglas de negocio esenciales del flujo crítico.
- API REST estable para el flujo principal y documentación de ejemplos de request/response.
- React web: flujo principal completo con formularios, validaciones, carga, errores y confirmaciones.
- React Native: al menos un flujo operativo completo, no una maqueta; debe crear/actualizar/consultar datos reales.
- Pruebas unitarias de reglas y al menos una prueba de integración del flujo.
- Docker/Compose funcional para PostgreSQL y aplicación o entorno equivalente reproducible.
- GitHub Actions inicial: build y tests automáticos.
- Informe breve de defectos encontrados, decisiones tomadas y deuda técnica.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE III - Parcial 2: cobertura funcional, seguridad y calidad

- Ampliación del modelo de datos y migraciones para cubrir los módulos principales restantes.
- Autenticación/autorización por roles aplicada a endpoints y acciones sensibles.
- Manejo centralizado de errores, validación consistente y códigos HTTP correctos.

- Web: dashboard/listados/filtros y al menos dos flujos de negocio adicionales conectados.
- Móvil: múltiples casos de uso del rol móvil principal, con estados de carga, error y refresco.
- Pruebas de integración adicionales y cobertura explícita de escenarios negativos.
- Auditoría básica de operaciones críticas definidas para el proyecto.
- Pipeline CI estable y política mínima de pull requests/revisión.
- Documentación de arquitectura actualizada y ADR de al menos dos decisiones relevantes.
- Demostración integrada con datos consistentes y sin intervención manual en base durante la defensa.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

PARTE IV - Examen Final: MVP completo, operación e integración

- Cumplimiento del alcance MVP obligatorio del proyecto y cierre de requisitos pendientes.
- Base PostgreSQL migrable desde cero y dataset de demostración reproducible.
- Backend estable con reglas de negocio, autorización, auditoría, errores y pruebas de los flujos críticos.
- Aplicación web completa para los roles administrativos/operativos definidos.
- Aplicación móvil completa para los roles móviles definidos, consumiendo la misma API.
- Docker/Compose y CI listos para demostrar build y ejecución reproducibles.
- Observabilidad básica: logs útiles, seguimiento de errores y evidencia de diagnóstico de una falla controlada.
- Spring AI integrado sólo en el caso de uso acotado definido, con fallback o manejo seguro de indisponibilidad.
- Manual técnico, manual breve de usuario, catálogo final de endpoints, modelo de datos y matriz final de trazabilidad.
- Defensa individual integral: cada estudiante debe explicar y modificar partes del sistema correspondientes al alcance final, independientemente de quién las haya implementado inicialmente.

Evidencia mínima de MVP del corte: el docente debe poder ejecutar una demostración funcional con datos persistidos y observar interacción real entre PostgreSQL, backend, web y móvil en el nivel alcanzado.

O. Preguntas de defensa individual sugeridas

- ¿Cómo evita contratos activos superpuestos?
- ¿Cómo modela Cuota y PagoAplicacion?
- ¿Por qué no conviene sobrescribir importes históricos?
- ¿Cómo una incidencia se transforma en orden?
- ¿Qué ve el técnico móvil y qué no debe ver?
- ¿Qué parte quedaría fuera por ser contabilidad formal?
- Identifique en código una regla de negocio y siga su recorrido desde el endpoint hasta persistencia.
- Cambie en vivo una validación sencilla solicitada por el docente y explique qué pruebas deberían actualizarse.
- Explique una decisión de modelado y una alternativa que rechazó el equipo.
- Muestre un error real controlado desde móvil o web hasta la respuesta del backend.

21. Matriz docente de equivalencia de complejidad

Proyecto	Modelo	RF	RN	Cientes	Complejidad central	Spring AI
----------	--------	----	----	---------	---------------------	-----------

11 FarmaStock 360	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
12 BiblioSmart	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
13 CargoTrack	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
14 SalonPro	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
15 EquipRent	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
16 AgroControl	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
17 BuildTrack	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
18 CampusLab	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
19 ONGImpact	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada
20 InmoRent	>= 12 entidades	>= 18 RF	10 RN	Web + móvil	Flujo transaccional + estados	IA acotada

22. Definition of Done común por historia funcional

- Criterios de aceptación comprendidos y trazables.
- Modelo de datos/migración actualizado si aplica.
- Regla implementada en la capa adecuada.
- Endpoint con validación y errores coherentes.
- Cliente web o móvil integrado cuando corresponda.
- Pruebas automatizadas relevantes en verde.
- Revisión mediante pull request.
- Sin secretos ni datos sensibles en repositorio/logs.
- Documentación mínima actualizada.
- Historia demostrable con datos persistidos.

23. Lista de control docente por entrega

Dimensión	Preguntas de control
Requisitos	¿El equipo puede demostrar cada requisito incluido y explicar sus criterios de aceptación?

Datos	¿El modelo representa el dominio, posee restricciones y puede recrearse por migraciones?
Backend	¿Las reglas viven en backend y están separadas de detalles de infraestructura?
API	¿Contratos, errores y códigos HTTP son coherentes y consumibles por dos clientes?
Web	¿Existe funcionalidad real, no sólo maquetación?
Móvil	¿Existe un caso de uso real de movilidad integrado al backend?
Pruebas	¿Se prueban reglas y escenarios negativos del corte?
Git/CI	¿Existe evidencia de colaboración, PRs y automatización?
Documentación	¿Coincide con lo implementado y permite trazabilidad?
Defensa	¿Cada estudiante entiende todo el alcance evaluable y puede modificar/diagnosticar partes del sistema?